

# FUNCTIONAL DESIGN SPECIFICATION

## NIT Bank Linux Process Operations Lab

Process Monitoring, Signals, Priority, Resilience and Incident Recovery

| Document Field            | Value                                                                                               |
|---------------------------|-----------------------------------------------------------------------------------------------------|
| Document Type             | Functional Design Specification (FDS)                                                               |
| Project                   | NIT Bank Linux Process Operations Lab                                                               |
| Customer / Business Owner | NIT Bank Operations Training                                                                        |
| Delivery Organization     | Nexus Institute of Technology                                                                       |
| Version                   | Revision 01                                                                                         |
| Date                      | 21 July 2026                                                                                        |
| Status                    | For Training and Design Review                                                                      |
| Purpose                   | Define how the approved Linux process project will be implemented, tested, controlled and accepted. |

## Document Control

| Version | Date        | Author / Owner       | Description                   | Status |
|---------|-------------|----------------------|-------------------------------|--------|
| 0.1     | 21-Jul-2026 | Project Controls     | Initial training design draft | Draft  |
| 1.0     | 21-Jul-2026 | Linux Technical Lead | Issued for classroom review   | Review |

## Approvals

| Role                         | Name | Signature | Date |
|------------------------------|------|-----------|------|
| Prepared By                  | TBD  |           |      |
| Reviewed By                  | TBD  |           |      |
| Project Manager / Instructor | TBD  |           |      |
| Business Owner / Sponsor     | TBD  |           |      |

## Reference Documents

| Reference         | Description                                                 |
|-------------------|-------------------------------------------------------------|
| CTR               | Cost, Time and Resource baseline                            |
| PMP               | Project governance, organization, risk and quality controls |
| Master Schedule   | 15-working-day baseline and milestones                      |
| NCD               | Implementation, validation and rollback runbook             |
| ITQ Register      | Open technical and business decisions                       |
| Student Lab Guide | Detailed root-account commands and validation steps         |

## Table of Contents

1. Executive Summary
2. Purpose and Objectives
3. Scope
4. Requirements Summary
5. Proposed Architecture
6. Functional Design
7. Process Flow and Operating Logic
8. systemd Service Design
9. Monitoring, Evidence and Incident Management
10. Security and Access Control
11. Availability, Performance and Capacity
12. Operational Design
13. Testing and Acceptance
14. Assumptions, Dependencies and Exclusions
15. Implementation Deliverables
- Appendix A – Example Configuration Logic
- Appendix B – Open Decisions and Customer Inputs

# 1. Executive Summary

This Functional Design Specification defines a controlled classroom solution that teaches Linux process management through a bank-operations scenario. Each student receives an assigned Rocky Linux 9 virtual machine in Xen Orchestra and acts as a junior Linux operations engineer responsible for building, monitoring, controlling and recovering a simulated report-generation service.

The project combines technical execution with business project controls. Students must work from approved CTR, PMP, Master Schedule, NCD and ITQ documents; create evidence; follow Go / No-Go gates; use rollback procedures; and obtain formal acceptance. The technical scope includes foreground and background jobs, PID and PPID analysis, process signals, nice and renice, nohup, /proc, parent-child relationships, zombie processes, systemd service management and a controlled high-CPU incident.

## 2. Purpose and Objectives

The purpose of this document is to translate the training and business requirements into an implementable and testable functional design.

- Create a repeatable process-management lab on one Rocky Linux 9 VM per student.
- Teach students to identify process owners, states, PID, PPID, CPU, memory and priority.
- Demonstrate safe use of SIGSTOP, SIGCONT, SIGTERM and SIGKILL.
- Demonstrate foreground/background job control, nohup and process persistence.
- Inspect process details through the /proc virtual filesystem.
- Create parent-child and zombie process demonstrations.
- Deploy a least-privilege systemd service that restarts after failure.
- Simulate a bank operational incident and require evidence-based recovery.
- Connect technical delivery with scope, schedule, cost, risk, change, quality and acceptance.

## 3. Scope

### 3.1 In Scope

- One assigned Rocky Linux 9 VM per student hosted in Xen Orchestra.
- Root-account verification and approved package installation.
- Creation of /opt/process-lab and the approved scripts and C program.
- Use of ps, top, pgrep, pkill, pstree, jobs, fg, bg, kill, nice, renice and nohup.
- Inspection of /proc/<PID>/status, cmdline, fd, limits, cwd and executable.
- Creation of reportsvc and report-generator.service.
- Technical validation, incident simulation, evidence collection, handover and closure.

### 3.2 Out of Scope Unless Added Through Change Control

- Production bank servers, customer accounts, transaction data or production credentials.
- High availability, disaster recovery, enterprise monitoring or SIEM integration.
- Internet-facing services, public ports or changes to shared network infrastructure.
- Application-development performance tuning beyond the controlled training processes.
- 24x7 managed support or production incident response.

## 4. Requirements Summary

| ID     | Requirement                                    | Design Response                  | Deliverable |
|--------|------------------------------------------------|----------------------------------|-------------|
| FR-001 | Verify assigned VM and root access             | whoami/id and inventory evidence | D1          |
| FR-002 | Create a long-running report-generator process | Bash script and log output       | D2          |
| FR-003 | Manage foreground and                          | jobs, fg, bg and state evidence  | D2          |

|        |                                            |                                              |    |
|--------|--------------------------------------------|----------------------------------------------|----|
|        | background jobs                            |                                              |    |
| FR-004 | Control processes with Linux signals       | STOP, CONT, TERM and KILL tests              | D3 |
| FR-005 | Simulate and resolve high CPU usage        | CPU worker, top/ps and remediation           | D4 |
| FR-006 | Manage process priority                    | nice and renice validation                   | D4 |
| FR-007 | Keep a process alive after logout          | nohup persistence test                       | D5 |
| FR-008 | Inspect process internals                  | /proc evidence                               | D6 |
| FR-009 | Demonstrate parent-child and zombie states | ps tree and Z-state evidence                 | D7 |
| FR-010 | Deploy a least-privilege systemd service   | reportsvc and unit file                      | D8 |
| FR-011 | Restart the service after failure          | Restart=on-failure test                      | D8 |
| FR-012 | Validate and close the project             | Validation log, incident report and sign-off | D9 |

## 5. Proposed Architecture

The logical context separates business control, virtualization, the student VM, the simulated application processes and the evidence/acceptance layer. The bank scenario is synthetic and contains no real customer data.

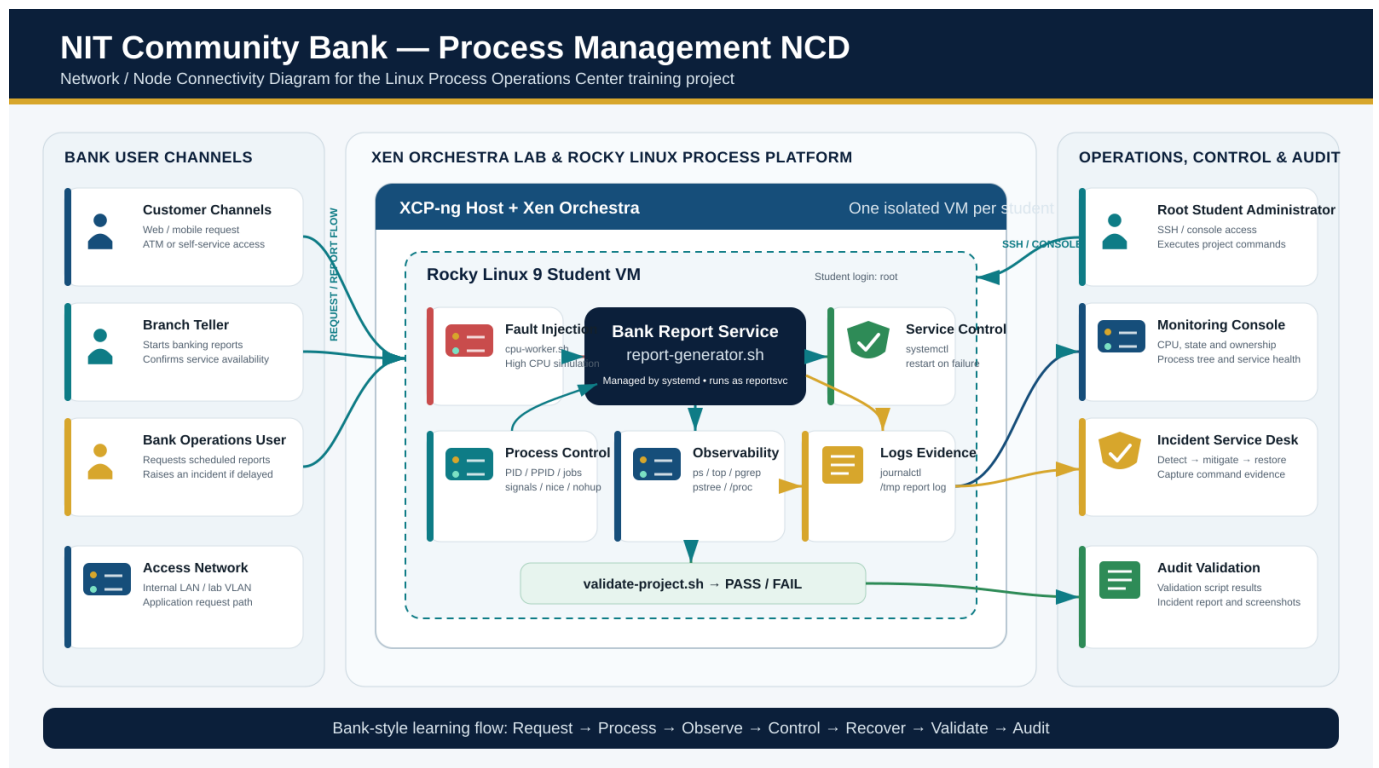


Figure 1 – Bank-style Linux Process Project Context Diagram

| Component                      | Primary Function                                                               |
|--------------------------------|--------------------------------------------------------------------------------|
| Business Sponsor / Instructor  | Approves objective, scope, Go / No-Go and final acceptance.                    |
| Xen Orchestra                  | Hosts student VMs, provides console access and pre-change snapshots.           |
| Rocky Linux 9 Student VM       | Provides the process, systemd, /proc and filesystem platform.                  |
| Report Generator               | Long-running simulated bank reporting process that writes timestamped logs.    |
| CPU Worker                     | Controlled fault process used to demonstrate high CPU and priority management. |
| Parent-Child / Zombie Programs | Demonstrate hierarchy, PPID and zombie state.                                  |
| systemd / reportsvc            | Provides least-privilege service execution and automatic restart.              |

|                     |                                                                              |
|---------------------|------------------------------------------------------------------------------|
| Evidence Repository | Stores command output, screenshots, validation results and incident reports. |
|---------------------|------------------------------------------------------------------------------|

## 6. Functional Design

### 6.1 Project Directory and Files

The approved project root is /opt/process-lab. The following controlled files will be created:

| File                                         | Purpose                                            | Owner / Mode         |
|----------------------------------------------|----------------------------------------------------|----------------------|
| report-generator.sh                          | Long-running report process and service executable | reportsvc / 0755     |
| cpu-worker.sh                                | Controlled high-CPU fault process                  | root / 0755          |
| parent-child.sh                              | Parent and child process demonstration             | root / 0755          |
| zombie.c / zombie                            | Zombie process source and compiled binary          | root / 0644 and 0755 |
| validate-project.sh                          | Automated acceptance checker                       | root / 0755          |
| /etc/systemd/system/report-generator.service | systemd unit                                       | root:root / 0644     |

### 6.2 Report Generator Behavior

- Write a start record containing date, PID and PPID to /tmp/report-generator.log.
- Write a timestamped report-generation record every ten seconds.
- Trap SIGTERM and SIGINT so the service can log a clean shutdown.
- Run continuously until stopped by an approved signal or systemd action.

### 6.3 CPU Worker Behavior

The CPU worker executes a tight loop and intentionally consumes one CPU core. Only one worker should normally run. The process exists solely to teach detection, priority change and safe termination.

### 6.4 Process-State Demonstrations

| State / Concept | Design Method                     | Expected Evidence                    |
|-----------------|-----------------------------------|--------------------------------------|
| Foreground      | Run report-generator directly     | Terminal occupied and PID visible    |
| Background      | Append & and use jobs -l          | Job number and PID                   |
| Stopped         | SIGSTOP or Ctrl+Z                 | STAT includes T                      |
| Sleeping        | Report generator waiting on sleep | STAT normally includes S             |
| Running         | CPU worker busy loop              | High CPU and R state may be observed |
| Zombie          | Child exits while parent sleeps   | STAT includes Z and <defunct>        |

## 7. Process Flow and Operating Logic

1. Business sponsor approves the project objective, scope, schedule and training assumptions.
2. Project Manager confirms VM assignment, root access, repositories, evidence location and snapshot readiness.
3. Student creates the approved scripts and validates syntax before execution.
4. Student identifies processes using ps, pgrep, top and pstree.
5. Student applies job control, process signals and priority changes and records results.
6. Student creates the systemd service and validates least-privilege execution.
7. Instructor starts the controlled high-CPU fault process.
8. Student diagnoses impact, changes priority, terminates the fault and restores the service.
9. Student runs automated validation, completes the incident report and obtains acceptance.
10. If a mandatory validation fails, the Project Manager authorizes correction or rollback.

## 8. systemd Service Design

### 8.1 Service Account

A dedicated system account named `reportsvc` will own and execute the service. The account uses `/sbin/nologin` and does not require an interactive password. This demonstrates least privilege even though students use `root` for classroom administration.

### 8.2 Unit Requirements

| Parameter      | Required Value / Behavior            |
|----------------|--------------------------------------|
| Description    | NIT Bank Report Generator            |
| Type           | simple                               |
| User / Group   | reportsvc / reportsvc                |
| ExecStart      | /opt/process-lab/report-generator.sh |
| Restart        | on-failure                           |
| RestartSec     | 5 seconds                            |
| TimeoutStopSec | 15 seconds                           |
| WantedBy       | multi-user.target                    |

### 8.3 Failure-Recovery Test

The MainPID is recorded, then the service process is terminated with `SIGKILL`. After the configured restart delay, `systemd` must return the service to active status with a different PID. This test proves the restart policy and provides evidence of resilience.

## 9. Monitoring, Evidence and Incident Management

### 9.1 Required Operational Views

- uptime and load averages
- `top` or `ps` sorted by CPU
- process owner, PID, PPID, STAT, NI, PRI, CPU and memory
- `systemctl` status and MainPID
- `journalctl` and `/tmp/report-generator.log`
- `/proc` process details
- Xen Orchestra snapshot and VM inventory

### 9.2 Incident Scenario

The simulated NIT Bank report platform becomes slow because `cpu-worker.sh` consumes a CPU core. The student acts as the Linux operations engineer and must identify the fault, preserve evidence, lower the priority, terminate it safely, verify the report-generator service and document the recovery.

### 9.3 Incident Record Minimum Content

- Incident summary and business impact
- Initial symptoms and timestamps
- Investigation commands and evidence
- Root cause
- Corrective actions
- Validation of recovery
- Preventive recommendations
- Final status and approvals

## 10. Security and Access Control

- Direct root access is permitted only because the environment is an isolated training VM.

- Students must not reuse the training root password outside the lab or share it in evidence.
- Each student may modify only the assigned VM.
- The report service must run as reportsvc rather than root.
- No real bank, customer, account or transaction data is permitted.
- Evidence must not include passwords, private keys or unrelated environment variables.
- Snapshots and destructive rollback commands require instructor or lab-administrator approval.

## 11. Availability, Performance and Capacity

### 11.1 Baseline VM

| Resource               | Recommended Classroom Baseline                   |
|------------------------|--------------------------------------------------|
| vCPU                   | 1                                                |
| RAM                    | 1 GB minimum; 2 GB preferred                     |
| Disk                   | 20 GB                                            |
| Operating System       | Rocky Linux 9                                    |
| Network                | SSH or Xen Orchestra console                     |
| Concurrent CPU Workers | One per student VM for a short controlled period |

### 11.2 Performance Control

The CPU worker must be stopped immediately after evidence is captured. The instructor may stagger the exercise when many VMs share one physical host. VM and host load should be observed to prevent the classroom fault from affecting unrelated services.

### 11.3 Availability Limitations

The baseline uses one standalone VM per student. VM failure is mitigated through Xen Orchestra snapshots or rebuild. High availability and disaster recovery are not included.

## 12. Operational Design

| Operational Task            | Method / Frequency                           |
|-----------------------------|----------------------------------------------|
| VM identity check           | Before every lab session                     |
| Snapshot verification       | Before implementation                        |
| Script syntax validation    | Before each execution                        |
| Process and load monitoring | During every process test                    |
| Service health check        | After start, restart and failure test        |
| Evidence update             | Immediately after each validation step       |
| ITQ review                  | Twice weekly until critical items are closed |
| Schedule and risk review    | Weekly                                       |
| Project closure review      | After all acceptance criteria pass           |

### 12.1 Runbook Content

- Verify VM and root access
- Install and validate packages
- Create scripts and service
- Perform job, signal, priority and /proc tests
- Create and remove the zombie process
- Execute incident diagnosis and recovery
- Run automated validation
- Perform manual cleanup or snapshot rollback
- Complete evidence and sign-off

## 13. Testing and Acceptance

| Test ID | Test Scenario                     | Expected Result                                 |
|---------|-----------------------------------|-------------------------------------------------|
| T-001   | Root access and assigned VM       | whoami=root and inventory matches               |
| T-002   | Foreground and background process | PID/job evidence recorded                       |
| T-003   | SIGSTOP and SIGCONT               | Process pauses and resumes                      |
| T-004   | SIGTERM and SIGKILL               | Expected termination behavior                   |
| T-005   | High CPU identification           | CPU worker appears in monitoring output         |
| T-006   | nice / renice                     | Expected NI values observed                     |
| T-007   | nohup persistence                 | Process survives logout                         |
| T-008   | /proc inspection                  | Required fields and file descriptors recorded   |
| T-009   | Parent-child hierarchy            | pstree shows expected relationship              |
| T-010   | Zombie process                    | Z state observed and removed through parent     |
| T-011   | systemd active and enabled        | Service healthy after restart                   |
| T-012   | Restart on failure                | New MainPID after forced failure                |
| T-013   | Least privilege                   | Service process owner is reportsvc              |
| T-014   | Incident recovery                 | CPU worker stopped and report service restored  |
| T-015   | Automated validator               | PROJECT VALIDATION SUCCESSFUL                   |
| T-016   | Business acceptance               | Evidence, incident report and sign-off complete |

### 13.1 Acceptance Criteria

- All mandatory runbook steps are Complete or formally accepted as Not Applicable.
- No uncontrolled CPU worker or lab zombie remains.
- report-generator.service is active, enabled and owned by reportsvc.
- Required process states, signals, priority and /proc evidence are submitted.
- The incident report explains business impact, root cause, corrective action and validation.
- The automated validation script reports success.
- Instructor / Project Manager signs technical and student-deliverable acceptance.

## 14. Assumptions, Dependencies and Exclusions

- The institute provides one supported Rocky Linux 9 VM per student.
- Students receive secure root credentials and work only on assigned VMs.
- Xen Orchestra console and snapshot support are available.
- Required package repositories are available during the project.
- The class uses synthetic NIT Bank data and scenarios only.
- Student evidence is stored in an approved LMS, Drive or project folder.
- Changes after design sign-off follow the NCD and ITQ/change-control process.
- Actual schedule and cost depend on final class size and instructor availability.

## 15. Implementation Deliverables

| Deliverable             | Content                                                              |
|-------------------------|----------------------------------------------------------------------|
| CTR                     | Approved scope, time, resources, cost and assumptions                |
| PMP                     | Governance, organization, RACI, risk, quality and communication plan |
| Master Schedule         | WBS, dates, milestones, progress tracker and Gantt chart             |
| NCD                     | Implementation, validation, evidence, approval and rollback runbook  |
| ITQ Register            | Open decisions, owners, due dates and closure evidence               |
| FDS                     | Approved functional and acceptance design                            |
| Student Technical Build | Scripts, C program and systemd service                               |
| Validation Evidence     | Test results, screenshots and command outputs                        |
| Incident Report         | Root cause, correction, recovery and prevention                      |
| Presentation / Handover | Student explanation of technical and business delivery               |

## Appendix A – Example Configuration Logic

### Report Generator Service Unit

```
[Unit]
Description=NIT Bank Report Generator
After=network.target

[Service]
Type=simple
User=reportsvc
Group=reportsvc
ExecStart=/opt/process-lab/report-generator.sh
Restart=on-failure
RestartSec=5
TimeoutStopSec=15

[Install]
WantedBy=multi-user.target
```

### Validation Logic

```
systemctl is-active report-generator.service
systemctl is-enabled report-generator.service
SERVICE_PID=$(systemctl show report-generator.service --property=MainPID --value)
ps -o user,pid,ppid,stat,etime,cmd -p "$SERVICE_PID"
pgrep -af cpu-worker
```

## Appendix B – Open Decisions and Customer Inputs

| Decision / Input                             | Required From        | Status |
|----------------------------------------------|----------------------|--------|
| Final student count and class structure      | Project Manager      | Open   |
| VM allocation and resource baseline          | Lab Administrator    | Open   |
| Credential distribution process              | Lab Administrator    | Open   |
| Snapshot retention and cleanup policy        | Lab Administrator    | Open   |
| Evidence repository and naming convention    | Document Controller  | Open   |
| Instructor and technical escalation contacts | Project Manager      | Open   |
| Grading pass mark and acceptance rules       | Instructor           | Open   |
| Final project dates and working hours        | Project Sponsor / PM | Open   |
| Business acceptance approver                 | Project Sponsor      | Open   |